TFT sector rotation Transformer-3k

Command-Line Interface Reference

Complete documentation of every command-line argument for all Python programs in this project — from data download and model training through paper trading simulation and live Schwab API trading.

Project Overview

This project implements a Temporal Fusion Transformer (TFT) model for weekly sector rotation trading. The model predicts which of the 11 SPDR sector ETFs will outperform or underperform over the next week, then executes a long/short options strategy (buy calls on top-3, buy puts on bottom-3).

The typical end-to-end workflow:

```
python run.py                    # download + preprocess + train +
evaluate

python predict.py                # generate this week's picks

python simulation.py setup       # start paper trading

python trade.py setup            # or start live trading via Schwab API
```

Scripts are organized into three tiers: Pipeline (run.py, download_data.py, train.py, evaluate.py), Analysis (predict.py, walk_forward.py, ensemble.py, generate_picks.py, plot_performance.py), and Trading (simulation.py, trade.py).

1 Pipeline Scripts

1.1 run.py — Full Pipeline

Orchestrates the complete pipeline: download market data, preprocess features, train the TFT model, and generate evaluation plots. This is the single command to run the project from scratch. Individual steps can be skipped when you only need to re-run part of the pipeline (e.g., re-training with new hyperparameters without re-downloading data).

Usage examples:

```
python run.py

python run.py --skip-download

python run.py --skip-preprocess --epochs 200 --device cuda
```

Argument	Type	Default	Description
`--skip-download`	flag	off	Skip Step 1 (data download). Use existing raw data in data/raw/. Implies --skip-preprocess is NOT set — preprocessing still runs unless you also pass --skip-preprocess.
`--skip-preprocess`	flag	off	Skip Step 1 (download) AND Step 2 (preprocessing). Jumps straight to training using whatever is already in data/processed/. Useful when only the model or hyperparameters have changed.
`--epochs`	int	config default (100)	Override the number of training epochs defined in config.py. Passed directly to the trainer. Higher values may improve accuracy at the cost of longer training time; early stopping will still terminate training early if the validation loss stops improving.
`--device`	string	auto	Override the compute device. Options: 'auto' (use GPU if available, otherwise CPU), 'cpu',

			'cuda', 'cuda:0', 'cuda:1', etc. The 'auto' default is usually the right choice.

1.2 download_data.py — Data Download

Downloads weekly OHLCV (Open, High, Low, Close, Volume) data for all 11 sector ETFs plus SPY from Yahoo Finance via yfinance. Data is saved as a Parquet file at data/raw/weekly_sector_data.parquet. The date range is taken from config.py but can be overridden on the command line.

Usage examples:

```
python download_data.py

python download_data.py --start-date 2015-01-01 --end-date 2024-12-31
```

Argument	Type	Default	Description
`--start-date`	string (YYYY-MM-DD)	config default (2010-01-01)	Start of the download window. All weekly bars on or after this date will be included. Earlier start dates provide more training data but older data may be less representative of current market structure.
`--end-date`	string (YYYY-MM-DD)	config default (2026-04-01)	End of the download window (exclusive — the last bar included is the last complete week before this date). Set this to yesterday or today to include the most recent data.

1.3 train.py — Model Training

Trains the Temporal Fusion Transformer on preprocessed sector data. Splits data into train (70%) / validation (15%) / test (15%) sets. Saves the best checkpoint (by validation loss) to checkpoints/best_model.pt and writes training results + test predictions to results/. Early stopping prevents overfitting.

Usage examples:

```
python train.py
```

```
python train.py --epochs 200 --lr 5e-4 --device cuda
```

Argument	Type	Default	Description
`--epochs`	int	config default (100)	Maximum number of training epochs. Training may stop before this if early stopping triggers (patience=15 epochs without validation loss improvement by default). Increase for potentially better results; decrease for faster experimentation.
`--lr`	float	config default (0.001)	Learning rate for the AdamW optimizer. The default 1e-3 works well for most runs. If training is unstable (loss spikes), try 5e-4 or 1e-4. If convergence is very slow, try 2e-3.
`--device`	string	auto	Compute device. 'auto' selects CUDA if available, otherwise CPU. Explicitly set to 'cpu' to force CPU-only training, or 'cuda' / 'cuda:N' for a specific GPU.

1.4 evaluate.py — Evaluation & Plots

Reads the saved training results and test predictions, then generates visualization charts saved to the results/ directory: training_curves.png (loss, rank correlation, top-3 accuracy over epochs) and cumulative_returns.png (strategy P&L vs SPY, per-sector correlation, sector allocation heatmap).

Usage examples:

```
python evaluate.py

python evaluate.py --max-days 52

python evaluate.py --results-dir results/walk_forward
```

Argument	Type	Default	Description
`--results-dir`	string	config default (results/)	Path to the directory containing training_results.json and

			test_predictions.npz. Override this to evaluate results from a different run (e.g., a walk-forward experiment saved to a different folder).
`--max-days`	int	0 (all)	Limit the strategy simulation to the first N test weeks. Useful for examining performance during a specific sub-period without re-training. 0 means use all available test weeks.

2 Analysis Scripts

2.1 predict.py — Weekly Sector Picks

Loads the trained model, downloads the most recent weeks of market data, runs inference, and prints the top-3 long picks (sectors to buy calls on) and bottom-3 short picks (sectors to buy puts on) for the current week. Run this every Friday to get next week's trades.

Usage examples:

```
python predict.py

python predict.py --n-seeds 5
```

Argument	Type	Default	Description
`--n-seeds`	int	1	Number of random seeds to average predictions across. With n-seeds=1 (default) the model is run once. With n-seeds > 1, the same saved checkpoint is re-initialized with N different random seeds, run N times, and the predictions are averaged. This reduces variance in the output signal at the cost of N× inference time. Values of 3–5 offer a good stability/speed tradeoff.

2.2 walk_forward.py — Walk-Forward Validation

Performs rigorous out-of-sample validation using a rolling walk-forward methodology. Divides the full dataset into overlapping windows, trains a fresh model on each window's training split, and tests on the immediately following held-out period. Each fold uses its own normalization statistics (computed only from that fold's training data) to prevent data leakage. Produces walk_forward_results.png and cumulative_returns.png.

Usage examples:

```
python walk_forward.py

python walk_forward.py --train-weeks 300 --test-weeks 52 --n-seeds 3
```

Argument	Type	Default	Description
`--train-weeks`	int	400	Number of weeks in each fold's training window (approximately 8 years at the default). Longer windows expose the model to more historical regimes but leave fewer folds if your dataset is not long enough.
`--val-weeks`	int	52	Validation window size per fold in weeks (default: 1 year). The model uses this split for early stopping and checkpoint selection within each fold. Does not affect test metrics.
`--test-weeks`	int	26	Out-of-sample test window per fold (default: 26 weeks, ~6 months). Also controls the step size between folds — the window slides forward by this many weeks after each fold.
`--epochs`	int	80	Maximum training epochs per fold. Lower than the standalone train.py default (100) because walk-forward runs many folds and total compute time multiplies. Early stopping also applies.
`--patience`	int	12	Early stopping patience per fold — number of epochs without validation loss improvement before training halts for that fold.
`--n-seeds`	int	1	Number of random seeds to train and average per fold. Using n-seeds > 1 creates a mini-ensemble within each fold, reducing fold-level variance. Each fold then takes n-seeds × the per-seed training time.

`--device`	string	auto	Compute device ('auto', 'cpu', 'cuda', 'cuda:N').

2.3 ensemble.py — Multi-Seed Ensemble

Trains N independent models (each with a different random seed) on the same train/val/test split used by train.py, then averages their predictions. The ensemble typically outperforms any single seed due to variance reduction. Produces ensemble_results.png (cumulative returns, per-seed Sharpes, seed agreement heatmap) and cumulative_returns.png.

Usage examples:

```
python ensemble.py

python ensemble.py --n-seeds 10

python ensemble.py --seeds 42 100 200 300 --max-days 52
```

Argument	Type	Default	Description
`--n-seeds`	int	5	How many models to train and ensemble. More seeds produce a more stable signal but training time scales linearly. The default five seeds (42, 123, 456, 789, 1024) are a good starting point; beyond 10 seeds the marginal benefit typically diminishes.
`--seeds`	int list	auto (first N of preset list)	Explicit list of seed values to use, space-separated. When provided, --n-seeds is ignored and exactly these seeds are used. Example: --seeds 42 100 200 300. Use this to reproduce a specific ensemble run.
`--max-days`	int	0 (all)	Limit evaluation to the first N test weeks. Does not affect training — all models are still trained on the full split. Useful for comparing ensemble performance over a specific sub-period.

`--device`	string	auto	Compute device ('auto', 'cpu', 'cuda', 'cuda:N').

2.4 generate_picks.py — Generate picks.json for Website

Runs the trained model and writes picks.json — a lightweight JSON file consumed by the Preceptron MarketMaker website. The file contains the top-3 long picks, bottom-3 short picks, and predicted returns for all 11 sectors. Optionally uploads the file to an S3 bucket so the website always shows the current week's predictions.

Usage examples:

```
python generate_picks.py

python generate_picks.py --s3

python generate_picks.py --output weekly_picks.json --s3 --bucket
my-bucket --key picks.json
```

Argument	Type	Default	Description
`--output`	string	picks.json	Local file path to write the JSON output. Can be a relative or absolute path. The file is always written locally first before any S3 upload.
`--s3`	flag	off	If set, upload the output file to S3 after writing it locally. Requires boto3 (pip install boto3) and AWS credentials configured via 'aws configure' or environment variables. The IAM user needs s3:PutObject permission on the target bucket.
`--bucket`	string	preceptron.com	S3 bucket name to upload to. Only used when --s3 is set. The bucket must exist and the AWS credentials must have write access.
`--key`	string	picks.json	S3 object key (path within the bucket). Only used when --s3 is set. For example, --key data/picks.json would store the file at

			s3://bucket/data/picks.json.

2.5 plot_performance.py — Performance Chart

Reads simulation_portfolio.json produced by simulation.py and generates a dark-themed PNG performance chart with two panels: (1) equity curve of your portfolio vs SPY buy-and-hold scaled to the same starting capital, and (2) weekly P&L bars with dollar labels. Optionally uploads to S3 for display on the Preceptron website.

Usage examples:

```
python plot_performance.py

python plot_performance.py --output chart.png --upload
```

Argument	Type	Default	Description
`--output`	string	performance.png	Output file path for the generated chart. Must end in .png. The chart is rendered at 150 DPI and sized at 12×8.5 inches, suitable for web embedding.
`--upload`	flag	off	Upload the PNG to S3 after generating it locally. Requires boto3 and valid AWS credentials. Uploads with ContentType: image/png and Cache-Control: no-cache so the website always fetches the latest chart.
`--bucket`	string	preceptron.com	S3 bucket name for the upload. Only relevant when --upload is set.
`--key`	string	performance.png	S3 object key for the uploaded PNG. Only relevant when --upload is set.

3 Trading Scripts

3.1 simulation.py — Paper Trading Simulation

Simulates the sector rotation options strategy with real market data but no real money. Uses yfinance to fetch live option chains and look up current prices, but all 'trades' are recorded only in simulation_portfolio.json. Run this for weeks or months to track hypothetical performance before committing real capital.

The weekly workflow:

```
python simulation.py setup --capital 10000     # one-time setup
python simulation.py predict                   # Friday: get picks
python simulation.py execute                   # Monday: 'buy' options
python simulation.py status                    # any time: check P&L
python simulation.py close                     # Friday: close positions
python simulation.py week                      # or do it all at once
```

setup — Initialize Portfolio

Creates simulation_portfolio.json with the starting configuration. Must be run once before any other subcommand. Re-running setup will overwrite the existing portfolio, so use 'reset' first if you want a clean slate.

Argument	Type	Default	Description
`--capital`	float	10000	Starting cash balance in US dollars. This is the total buying power available to the simulation. Each week, up to 90% of available cash is allocated equally across new positions (10% kept as a buffer).
`--otm`	float	1.0	Strike price distance out-of-the-money in dollars. For a call on an ETF trading at $80, --otm 1.0 targets the $81 strike. For a put it targets $79. Smaller values (e.g. 0.50) give more delta/leverage;

			larger values (e.g. 2.0) are cheaper but require a bigger move to profit.
`--commissions`	float	0.0	Per-contract commission charged on both the buy leg and the sell leg, in dollars. For example, Schwab charges $0.65/contract. Setting this to a realistic value makes the simulation P&L more accurate. Each options contract represents 100 shares but commissions are per contract, not per share.

predict — Generate Picks

Runs the trained TFT model against the latest sector data and stores the picks in simulation_portfolio.json. Prints the top-3 long and bottom-3 short sectors with their predicted relative returns. No arguments beyond the subcommand name.

execute — Open Positions

Reads the most recent unexecuted picks and 'buys' the corresponding options at current market prices (mid of bid/ask, or ask if no bid). Closes any existing positions that are no longer in the current picks before opening new ones. Allocates cash equally across new positions. No arguments beyond the subcommand name.

close — Close All Positions

Looks up the current price for every open position and records the closing trade. If a live price cannot be fetched, estimates the exit price using a 3%/day theta-decay model. Records weekly P&L and resets positions to empty. No arguments beyond the subcommand name.

status — Portfolio Status

Prints the current portfolio: cash balance, open positions with unrealized P&L (mark-to-market at current bid/ask), realized P&L from closed trades, win rate, and best/worst week. No arguments beyond the subcommand name.

history — Trade History

Prints a table of every closed trade (entry date, ticker, type, strike, entry price, exit price, contracts, P&L) plus a weekly P&L summary. No arguments beyond the subcommand name.

week — Full Weekly Cycle

Convenience command that runs close → predict → execute in sequence. Useful on Fridays when you want to close this week's positions and immediately open next week's in one step. No arguments beyond the subcommand name.

reset — Reset Simulation

Deletes simulation_portfolio.json, clearing all history. Irreversible. Use before starting a fresh simulation run. No arguments beyond the subcommand name.

3.2 trade.py — Live Trading via Charles Schwab API

Executes the same sector rotation strategy as simulation.py but places real options orders through the Charles Schwab API (using the schwab-py library). On every run the program asks whether to use Sandbox mode (paper trading account) or Live mode (real account with real money). Live mode requires typing YES to confirm. State is saved to trade_portfolio.json.

One-time setup required before first use:

```
pip install schwab-py

# Create schwab_config.json with your API credentials

python trade.py setup --capital 10000 --stop-loss 5
```

The weekly workflow:

```
python trade.py predict           # Friday: get picks

python trade.py execute           # Monday: place real orders

python trade.py status            # any time: check positions +
early-close menu

python trade.py close             # Friday: close all positions
```

Mode selection can be bypassed with --mode:

```
python trade.py --mode sandbox status

python trade.py --mode live execute   # still requires YES confirmation
```

Global Option (applies to all subcommands)

Argument	Type	Default	Description
`--mode`	string	interactive prompt	Trading mode: 'sandbox' (paper trading account) or 'live' (real account). When omitted, the program always prompts at startup. When set to 'live', an additional YES confirmation is still required. Useful for scripting or automation where an interactive prompt is not possible.

setup — Initialize Trade Portfolio

Creates trade_portfolio.json. Must be run once before any other subcommand. Records the trading mode so subsequent commands know which account to use.

Argument	Type	Default	Description
`--capital`	float	10000	Starting cash balance in US dollars. Same semantics as simulation.py. Note that this tracks cash in trade_portfolio.json for record-keeping; actual buying power is determined by your Schwab account balance.
`--otm`	float	1.0	Strike price distance out-of-the-money in dollars. Passed to the Schwab option chain API (or yfinance fallback) to select the target strike. A $1 OTM call on an $80 ETF targets the $81 strike.
`--commissions`	float	0.0	Per-contract commission in dollars charged on each leg (buy AND sell), for bookkeeping purposes. Set to your actual broker

			rate (e.g. 0.65) so that recorded P&L matches your account statements.
`--stop-loss`	float	5.0	Stop loss percentage. After buying each option, a GTC (Good Till Cancelled) stop sell-to-close order is immediately placed at entry_price × (1 − stop_loss / 100). Example: if an option costs $3.00 and --stop-loss is 5, the stop order is placed at $2.85. Set to 0 to disable stop loss orders entirely. The stop order ID is stored in trade_portfolio.json and cancelled automatically when a position is closed manually or at week-end.

predict — Generate Picks

Identical to simulation.py predict: runs the TFT model and stores the picks in trade_portfolio.json. No arguments beyond the subcommand name.

execute — Place Real Orders

Reads the latest picks and places limit buy-to-open orders for each new position via the Schwab API. Immediately follows each buy with a GTC stop sell-to-close order at the configured stop loss price. Closes (via market sell) any positions no longer in the current picks and cancels their stop loss orders first. Order IDs are saved in trade_portfolio.json for tracking. No arguments beyond the subcommand name.

close — Close All Positions

Places market sell-to-close orders for every open position via the Schwab API, cancels all pending GTC stop loss orders, and records final P&L. No arguments beyond the subcommand name.

status — Portfolio Status with Interactive Menu

Shows the same portfolio summary as simulation.py status (cash, positions, unrealized/realized P&L, win rate). After the summary, prompts the user to open an interactive early-close menu.

The interactive menu (arrow-key driven, Windows):

Use ↑↓ arrow keys to navigate a list of open positions. Press Enter to select a position to close early — a y/N confirmation is requested before the order is placed. Multiple positions can be closed in one session. Select 'Done' or press Escape to exit the menu without further changes. On non-Windows systems, a plain-text numbered fallback is used instead. No arguments beyond the subcommand name.

history — Trade History

Same as simulation.py history but also shows the close reason column (weekly_close, rebalance, or early_close) so you can distinguish planned weekly closes from manual early exits. No arguments beyond the subcommand name.

week — Full Weekly Cycle

Runs close → predict → execute in sequence, placing real orders at each step. No arguments beyond the subcommand name.

reset — Reset Trade Portfolio

Deletes trade_portfolio.json. Does NOT cancel any open orders in your Schwab account — cancel those manually in the Schwab platform before resetting. No arguments beyond the subcommand name.

4 Configuration Files

4.1 config.py — Project Configuration (not a CLI script)

config.py is not run directly from the command line — it is imported by all other scripts. It defines the default values for data paths, feature engineering, model architecture, and training hyperparameters. Many of these can be overridden by CLI arguments in the scripts above.

Key config.py defaults:

Setting	Default	Override via
`data.start_date`	2010-01-01	download_data.py --start-date
`data.end_date`	2026-04-01	download_data.py --end-date
`model.lookback_len`	12 weeks	(edit config.py directly)
`model.d_model`	64	(edit config.py directly)
`model.n_heads`	4	(edit config.py directly)
`model.lstm_layers`	2	(edit config.py directly)
`model.dropout`	0.1	(edit config.py directly)
`train.epochs`	100	run.py / train.py --epochs
`train.learning_rate`	0.001	train.py --lr
`train.batch_size`	32	(edit config.py directly)
`train.patience`	15	(edit config.py directly)
`train.seed`	42	(edit config.py directly)
`train.device`	auto	run.py / train.py --device
`train.train_ratio`	0.70	(edit config.py directly)
`train.val_ratio`	0.15	(edit config.py directly)
`train.checkpoint_dir`	checkpoints/	(edit config.py directly)
`train.results_dir`	results/	evaluate.py --results-dir

4.2 schwab_config.json — Schwab API Credentials (trade.py only)

Created manually before first use of trade.py. Stores your Schwab developer credentials and account numbers. Keep this file out of version control (.gitignore it). If the file is missing, trade.py prints the required structure and exits.

Required fields:

Argument	Type	Default	Description
`api_key`	string	(your value)	App Key from developer.schwab.com. Also called 'Consumer Key' in older Schwab documentation. Obtained by creating an app at developer.schwab.com and clicking 'View Details'.

`app_secret`	string	(your value)	App Secret from the same Schwab developer portal page. Treat this like a password — never share it or commit it to version control.
`callback_url`	string	https://127.0.0.1	OAuth2 redirect URI registered when you created your Schwab app. Must match exactly. 'https://127.0.0.1' is the recommended value for local development since it does not require a running web server.
`sandbox_account`	string	(your value)	Schwab paper trading account number (8 digits). Used when sandbox mode is selected at startup. Accessible from the Schwab website under your StreetSmart Edge Paper Trading account.
`live_account`	string	(your value)	Real Schwab brokerage account number (8 digits). Used only when live mode is selected and confirmed. Real orders will be placed against this account.

5 Quick Reference Card

The table below lists every CLI argument across all scripts at a glance.

Script	Argument	Purpose (one line)
run.py	--skip-download	Skip data download step
run.py	--skip-preprocess	Skip download + preprocess steps
run.py	--epochs N	Override training epochs
run.py	--device D	Override compute device
download_data.py	--start-date DATE	Download start date (YYYY-MM-DD)
download_data.py	--end-date DATE	Download end date (YYYY-MM-DD)
train.py	--epochs N	Override training epochs
train.py	--lr F	Override learning rate
train.py	--device D	Override compute device
evaluate.py	--results-dir PATH	Path to results directory
evaluate.py	--max-days N	Limit evaluation to first N weeks
predict.py	--n-seeds N	Average predictions over N seeds
walk_forward.py	--train-weeks N	Training window size (weeks)
walk_forward.py	--val-weeks N	Validation window size (weeks)
walk_forward.py	--test-weeks N	Test window / fold step (weeks)
walk_forward.py	--epochs N	Epochs per fold
walk_forward.py	--patience N	Early stopping patience per fold
walk_forward.py	--n-seeds N	Seeds per fold (mini-ensemble)
walk_forward.py	--device D	Compute device
ensemble.py	--n-seeds N	Number of seeds to ensemble
ensemble.py	--seeds N [N ...]	Explicit seed list
ensemble.py	--max-days N	Limit evaluation to first N weeks
ensemble.py	--device D	Compute device
generate_picks.py	--output PATH	Output JSON file path
generate_picks.py	--s3	Also upload to S3
generate_picks.py	--bucket NAME	S3 bucket name
generate_picks.py	--key KEY	S3 object key
plot_performance.py	--output PATH	Output PNG path
plot_performance.py	--upload	Upload PNG to S3
plot_performance.py	--bucket NAME	S3 bucket name
plot_performance.py	--key KEY	S3 object key
simulation.py setup	--capital F	Starting cash ($)
simulation.py setup	--otm F	OTM strike distance ($)
simulation.py setup	--commissions F	Per-contract commission ($)
trade.py	--mode MODE	sandbox or live (skip prompt)
trade.py setup	--capital F	Starting cash ($)
trade.py setup	--otm F	OTM strike distance ($)
trade.py setup	--commissions F	Per-contract commission ($)
trade.py setup	--stop-loss F	Stop loss % (e.g. 5 = 5%)